



An iterative structure decomposition learning method for complex Bayesian networks

Xiaolong Jia^{1,2} · Hongru Li¹ 

Received: 9 October 2025 / Accepted: 1 March 2026
© The Author(s) 2026

Abstract

Bayesian network structure decomposition learning is an advanced method for building complex Bayesian network structure. The methods build an initial graph at one time, then decompose it and learn these sub-graphs, and finally combine all the subgraphs into a complete Bayesian network structure. However, the result is affected by the initial graph. The paper firstly proposes a novel structure decomposition learning method of complex Bayesian networks based on iterative learning. Firstly, we develop an iterative framework for complex Bayesian network structures decomposition learning. In each iteration, we continuously use the undirected graph contained in the learning result for new decomposition learning to optimize the learning result. Secondly, we propose three mutation operators to generate new initial graphs in iterative processes. Within our framework, mutation operators facilitate the generation of new initial graphs by strategically utilizing existing structural information in various manners, thereby enhancing the efficiency of the iterative learning process. Finally, numerous experiments have been conducted to demonstrate the learning process. Compared with advanced algorithms, our method outperforms non-iterative algorithms.

Keywords Bayesian network · Structure learning · Structure decomposition · Mutation operators · Iterative learning

Introduction

Bayesian network (BN) is one of uncertain knowledge representation and inference model combining graph theory and probability theory. Because of its strong logicity and interpretability [1], Bayesian network can provide a structured representation of domain knowledge [2]. It has been implemented in a variety of fields, such as bioinformatics, social security, industrial fault diagnosis [3, 4], risk analysis [5], etc. Currently, there is a rapid growth of big data in biomedical and various other fields, characterized by a high number of variables, typically in the hundreds or thousands within datasets. Bayesian network learning is a prerequisite for solving problems, including Bayesian network structure (BNs)

learning and conditional probability table (CPT) learning. However, learning an accurate Bayesian network structure from a limited sample is NP-hard [6] and more valuable than conditional probability table learning. Now, people have developed many network structure algorithms that learn a complete graph directly. They are generally divided into score-based (S&S) [7], constraint-based (CB), and their hybrid methods [8]. When learning complex Bayesian network structure, these methods will give very poor results. It is because the complexity of the CB methods is exponential and the results are unreliable [9, 10], while the S&S methods may get trapped into a local optimum [11–14]. It presents a serious challenge to learn the structure of complex Bayesian network.

Structure decomposition learning is a promising method to learn complex Bayesian network structure [1, 7, 15–19]. It introduces the divide-and-conquer strategy and divides the learning process into four stages, namely constructing initial graph, decomposing structure, learning subgraphs and combining subgraphs. Following the creation of an initial graph from observational data, the researchers utilize decomposition techniques such as maximal prime subgraph decomposition (MPD) [19–21], graph partition [7, 22], and

Xiaolong Jia and Hongru Li contributed equally to this work.

✉ Hongru Li
lihongru@ise.neu.edu.cn

¹ College of Information Science and Engineering, Northeastern University, Shenyang 110819, China

² School of Electrical and Information Engineering, Ningxia Institute of Science and Technology, Shizuishan 753000, China

community detection methods [1, 16, 23–26] to break down the initial graph into some small subgraphs, which are then used for the learning process. After these subgraphs are learned to be subnetworks, which are combined finally into a whole Bayesian network structure. The strategy not only reduces the complexity of learning, but also improves the accuracy of Bayesian network structures.

Generally, complex Bayesian networks are large, with a lot of nodes, sparse structure and community characteristics. In the communities, the connections within the communities are close and the connections between the communities are sparse [1, 18, 27–29]. Therefore, in recent years, many decomposition learning methods of complex Bayesian network structures have been explored according to the community characteristics [1, 16, 18, 30]. The methods based on community detection is to find the sub-communities of Bayesian network structure by clustering network nodes through k -means clustering, hierarchical clustering or optimal modularity. Given the prior information of the number of subgraphs k , the nodes in Bayesian network are divided into k clusters by k -means, so that the connections within clusters are tight and the connections between clusters are sparse. Li et al. [16] introduced the block learning algorithm BLMKM, which employed an enhanced k -means algorithm to group nodes in a Bayesian network and identified potential graph structures based on these separated blocks. The algorithm aimed to maximize the total cost, which was calculated as the sum of the mutual information between all nodes within each subset. Lee proposed a recursive clustering and order restriction (R-CORE) method [24]. For an undirected weighted complete graph, the objective was to construct clusters that aim to minimize the total sum of the degrees of the undirected edges connecting nodes within different clusters. In the method, the degree of an edge was the mutual information of two nodes. Hierarchical clustering is another main clustering method for Bayesian network structure decomposition. It mainly includes top-down splitting strategy and bottom-up aggregation strategy. From top to bottom, Shuo-hao et al. [25] divided the BN skeleton, which was obtained by testing the dependency between two variables, into some communities by GN algorithm. Gu and Zhou [26] proposed a PEF method. The method was a bottom-up hierarchical clustering with average linkage that automatically chooses the number of clusters. He et al. [1] introduced the idea of granularity calculation and proposed the method named SL-MGI to learn the structure of complex Bayesian network. The method decomposed an undirected independence graph by the average-link hierarchical clustering method to realize granulation. Modularity was used to measure the quality of community detection, which naturally transformed the community detection problem into an optimization problem. The Louvain community detection algorithm is a main community detection method based on modularity optimization.

Mandar et al. applied the Louvain community detection algorithm to find communities in a weighted undirected graph of Bayesian network [23].

In the Bayesian network decomposition learning methods, the construction of initial undirected graph is the first step of structural decomposition, representing the relationship between nodes. The initial graph directly affects the node division. If the initial graph is closely connected, the decomposed subgraph granularity is still large, which is not conducive to learning. If the initial graph is sparsely connected, some true edges may be lost in the sketch, which will reduce the partition selectivity and high coverage. It will affect the learning result of complex Bayesian network structure decomposition.

The initial graph is usually constructed by measuring the correlation between two nodes, including mutual information [31], conditional mutual information [32], Chi-square test [33], Pearson correlation coefficient [34], entropy [35], etc. Among them, mutual information and conditional mutual information are hypothesis-free methods that measure unknown, non-linear and complex correlations between nodes rather than linear correlations, and solve the problem of intensive calculation of parameters. Therefore, mutual information and conditional mutual information are often used to infer complex Bayesian networks. However, mutual information overestimates the relationship between nodes to some extent and fails to distinguish between indirectly related nodes and directly related nodes, resulting in additional connections. While conditional mutual information is able to distinguish indirectly related nodes from directly related nodes, it cannot derive the causal direction of nodes in the network. As the size of condition set increases (i.e., higher order) and the cumulative effect makes the conditional mutual information test unreliable [36], Type II errors can occur where correlations between directly related nodes are underestimated and connections between directly related nodes are lost. In complex networks, such errors are particularly prominent and even increase the computational cost of testing. The extra connections and lost connections in the initial diagram will change the independence, correlation and community characteristics of nodes in the real network and affect the structural decomposition results.

However, in the existing Bayesian network structure decomposition learning, the initial graph is constructed and decomposed at one time, and the initial graph generally includes superstructure, skeleton, moral graph and so on. Among them, the superstructure carries more extra connections to retain more real connections, and the skeleton strives to be close to the connections of the real Bayesian network structure, while the moral graph minimizes the sum of extra connections and lost connections, and increases the moral edge to retain the independent information of all conditions. No matter what kind of initial graph is obtained at one time,

there will be extra connections and lost connections due to the wrong test, which cannot reflect the node connections of the real network, which will affect the quality of decomposition learning.

Therefore, a novel structure decomposition learning method of complex Bayesian networks is innovatively proposed in the paper, which is an iterative method and a dynamic method of constructing initial graph for optimal learning results. Unlike the existing decomposition learning methods that are completed at once, this is an iterative learning approach that aims to achieve higher quality Bayesian networks. Our contributions are as follows:

Firstly, an iterative structure decomposition learning framework for complex Bayesian networks is proposed. In the method, a directed acyclic graph is obtained by decomposition learning from a random initial graph. Then the directed acyclic graph is transformed into a new initial graph and decomposition learning is carried out. The method updates the initial graph dynamically and performs decomposition learning in an iterative manner. To prevent the iterative learning from getting stuck in a local optimum, we design three parallel learning processes during the iteration. The initial graph and its variants with higher scores and closer to the real network in the learning results are used for new decomposition learning. Compared with the learning method of constructing initial graph at one time and decomposing it, the method is an iterative optimization method, which can dynamically improve the initial graph and improve the learning result of complex Bayesian network structure.

Secondly, we propose three mutation operators to generate new initial graphs in iterative processes. The first operator is SELECT, which uses the skeleton of the currently learned optimal directed acyclic graph for decomposition, which can get closer to the community information and independence information of the real network. The second is AND. AND retains the node connection information common to the currently learned DAGs with high score. The third is OR, which preserves more real or possibly real connection information of the currently learned DAGs with high score. In our framework, these mutation operators employ various strategies to leverage existing structural information, aiming to facilitate the creation of high-quality initial graphs, thereby improving the efficiency of the iterative learning process.

Finally, we have done a lot of experiments to demonstrate the learning process. At the same time, compared with several advanced algorithms, the results show that the method proposed in this paper has greater advantages.

The rest of the paper is organized as following. “[Related work](#)” provides an overview of the fundamental concepts and discusses existing works related to learning the structures of Bayesian networks through decomposition. “[Our method](#)” introduces the method we proposed. “[Experiment](#)” analyzes

the performers of the method. Finally, the conclusion and future work are introduced.

Related work

Bayesian network

Bayesian network $\mathcal{B}$ is defined as a binary group $(\mathcal{G}, \mathcal{P})$, where $\mathcal{G}$ is a directed acyclic graph (DAG) that encodes a joint probability distribution $\mathcal{P}$ over a vector of random variables $V = (X_1, X_2, \dots, X_n)$. Each node in the graph represents a variable in V . For convenience, we will use the terms “random variable” and “node” interchangeably throughout this paper. Unless otherwise specified, a non-bold capital letter represents a variable. The DAG can be represented as a vector $\mathcal{G} = (\mathbf{Pa}_1, \mathbf{Pa}_2, \dots, \mathbf{Pa}_n)$ where each $\mathbf{Pa}_i$ is a subset of $V \setminus \{X_i\}$ and specifies the parents of X_i in the graph. Each node in the DAG is associated with a conditional probability distribution $P(X_i | \mathbf{Pa}_i)$. Then the joint distribution $P(V)$ must be factorized as follows

$$P(V) = \prod_{i=1}^n P(X_i | \mathbf{Pa}_i) \quad (1)$$

The factorization formula (1) states that a variable is conditionally independent of each of its non-descendants, given its parents.

Figure 1 illustrates two representations of a Bayesian network structure with five nodes (X_1, X_2, X_3, X_4, X_5). In the Fig. 1a, the left panel shows the directed acyclic graph representation, where directed edges encode causal or dependency relationships between nodes, and the right panel presents its corresponding adjacency matrix, with entry (i, j) marked as 1 if there is a directed edge from X_i to X_j , and 0 otherwise. Figure 1b presents the corresponding undirected graph (skeleton) obtained by removing edge directions from the DAG, retaining only the adjacency relationships between nodes.

For a Bayesian network with n nodes, there are $f(n)$ candidate network structures.

$$f(n) = \sum_{i=1}^n (-1)^{i+1} \binom{n}{i} 2^{i(n-i)} f(n-i) \quad (2)$$

$$f(1) = 1, f(2) = 3, f(3) = 25, \dots, f(10) \approx 4.2 \times 10^{18}$$

When the number of nodes is small, it's feasible to explore all potential network structures to identify the optimal one. For a complex Bayesian network structure, there are dozens or even hundreds of nodes. The possible Bayesian network structure increases exponentially with the number of nodes, and finding the optimal network structure is NP-hard.

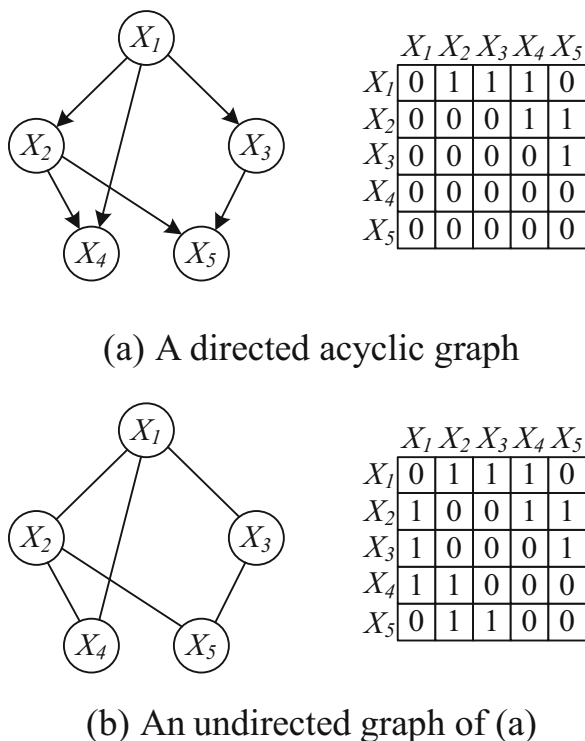


Fig. 1 Representation of Bayesian network structure

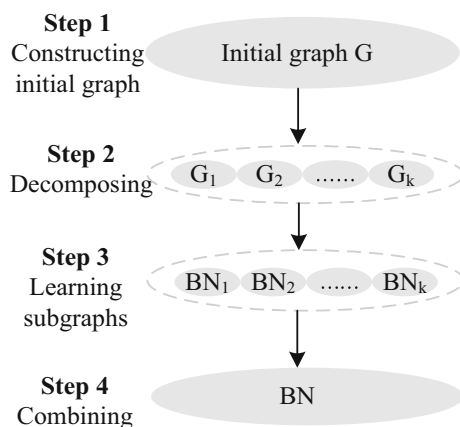


Fig. 2 The framework of Bayesian network structure decomposition learning

Bayesian network structure decomposition learning

Bayesian network structure decomposition learning employs a divide-and-conquer strategy and consists of several procedures to accomplish the learning task. This approach involves breaking down the complex network into smaller, more manageable subgraphs, which are then individually analyzed to determine their structures. The results from these subgraphs are subsequently combined to reconstruct the overall network structure. This method allows for more efficient and effective learning, particularly when dealing with large and complex networks. As shown in Fig. 2, it is a popular

framework, which divides the task into four processes. They are constructing initial graph, decomposing initial graph, learning subgraphs, and combining subgraphs. The initial graph is an undirected graph built directly from data, which implies dependence or conditional independence relationships between variables, and is usually obtained using mutual information or correlation tests to construct the initial graph. The initial graph decomposition is to cluster the variables in the Bayesian network using the graph decomposition method, that is, the initial graph is separated to small subgraphs, and each subgraph contains nodes with close local dependence in the Bayesian network structure. Then, the local Bayesian network structures can be learned through traditional global learning algorithm. Since the number of nodes in each local structure is small, more optimal local Bayesian network structures can be obtained in less time. Finally, local Bayesian network structures were recovered into a complete complex Bayesian network structure by combining rules and the observed data [7, 17].

Decomposition is achieving accurate clustering of Bayesian network nodes, which is an important guarantee for the accuracy of local structures identification and complete large Bayesian network structure learning. The effect of node clustering depends not only on the clustering method, but also on the quality of the initial graph. However, in the existing Bayesian network structure decomposition learning, the initial graph is constructed and decomposed at one time. There will be extra connections and lost connections due to the unreliable tests. It will make the decomposition and final DAG very poor when attempting to restore the original network structure.

Our method

The framework of the method

In this paper, we propose a novel Bayesian network structure decomposition learning method based on iterative learning, named *iter-DSLA*. The method employs the Bayesian network structure decomposition learning framework described in “[Bayesian network structure decomposition learning](#)”, and innovatively introduces the concept of iterative learning. The undirected graph corresponding to the DAG obtained from each decomposition learning is used as the input for the next iteration. Because each learned skeleton is closer to the actual network than the undirected graph learned before. Therefore, conducting decomposition learning on the new skeleton will make the learning results more similar to the actual network. After multiple iterations of decomposition learning, the results converge and an optimal Bayesian network structure is obtained. To avoid iterative learning falling into a local optimum, we design three operators to obtain

the new initial graphs. The following is the flow chart of our proposed algorithm based on iterative learning.

As can be seen from the Fig. 3, our method encompasses 6 steps, building constructing initial graph, decomposing, learning subgraph, combining, updating the optimal network, and exiting or producing new undirected graphs. The method performs three decomposition learning in parallel. At the beginning of the iteration, we randomly generate three BNs and convert them to initial graphs by $\text{to_G}()$ to avoid complicated undirected graph learning process. In each subsequent iteration, the Bayesian network structure learned from each decomposition is transformed into an undirected graph as the input of a new undirected graph. To prevent the algorithm from entering a local optimum, we also propose three mutation operators, SELECT, AND, OR. Three operators result in three different undirected graphs, which are input as undirected graphs learned by the new iterative decomposition. Therefore, we need to perform three decomposition learning in parallel. After each iteration of decomposition learning, we will obtain three new Bayesian networks structures. We score and select the three structures with the highest scores to retain. But these structures are not equivalence classes. If the number of iterations reaches a preset value, the decomposition learning is terminated. Otherwise, the undirected graphs of three newly preserved optimal networks are mutated and decomposition learning is carried out in the next iteration. The mutation operators facilitate the generation of new initial graphs by strategically utilizing existing structural information in various manners, thereby enhancing the efficiency of the iterative learning process. Therefore, through multiple iterations, the obtained Bayesian network will capture more and more accurate information, making the results increasingly closer to the actual network. In Fig. 3, $\text{to_G}()$ converts a directed acyclic graph of a Bayesian network into its corresponding undirected skeleton. Algorithm 1 is the pseudo-code for the graph transformation performed by $\text{to_G}()$, providing the details of the operation.

Algorithm 1 $\text{to_G}()$

Input: DAG
Output: G
<pre> n = size(DAG) G = zero_matrix(n, n) for i = 1:n for j = 1:n if DAG[i][j] == 1 G[i][j] = 1; G[j][i] = 1; else G[i][j] = 0 end end end return G </pre>

Three operators

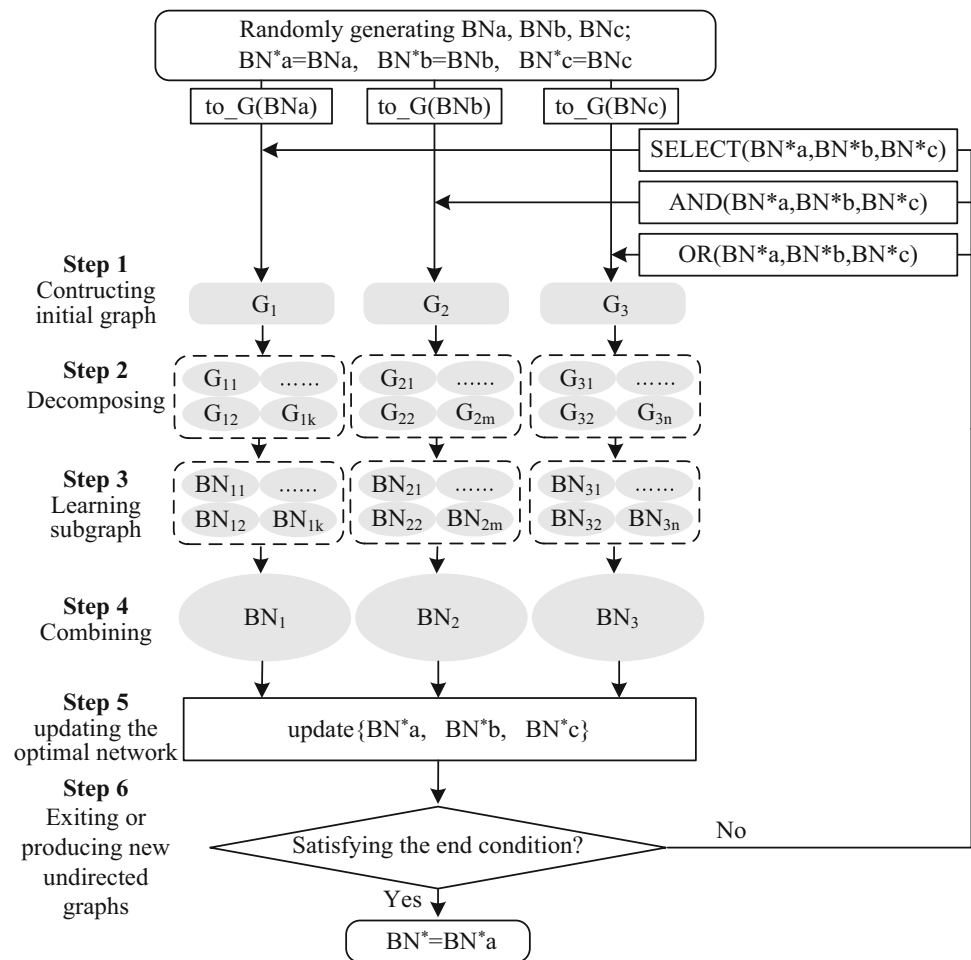
Three operators (SELECT, AND, OR) have been proposed in the paper to generate high-quality new initial graphs for iterative decomposition learning by strategically leveraging existing structural information from learned directed acyclic graphs, which not only avoids trapping the iterative process in local optima but also enhances the efficiency and accuracy of complex Bayesian network structure learning.

(1) SELECT operator

The first operator we proposed is called SELECT, which extracts the skeleton from the Bayesian network structure with the highest score as the initial graph to obtain a better Bayesian network structure in the subsequent decomposition learning. It finds the highest quality Bayesian network structure from all candidate Bayesian network structures by scoring algorithms. This structure fits the data the best and is regarded as the one most likely to represent the dependencies between the variables. The skeleton is an undirected graph after the direction is removed from the Bayesian network structure, which preserves the dependency between variables [37]. Because the highest-rated Bayesian network structure can more accurately represent the dependencies between variables, the corresponding skeleton graph also implies more real edges. Using the skeleton graph as the initial graph for decomposition learning can make full use of the dependency information in the Bayesian network structure with the highest score that has been learned, thus improving the accuracy of the Bayesian network structure obtained in the subsequent decomposition learning. Therefore, the SELECT operation can improve the accuracy of the Bayesian network structure obtained in the subsequent decomposition learning by using the dependency information provided by the Bayesian network structure with the highest score.

In Fig. 4, $\text{Score}(\text{BN}^*a) > \text{Score}(\text{BN}^*b) > \text{Score}(\text{BN}^*c)$, the BN^*a skeleton extracted by the SELECT operator is used as the undirected initial graph $G1$ for structure decomposition

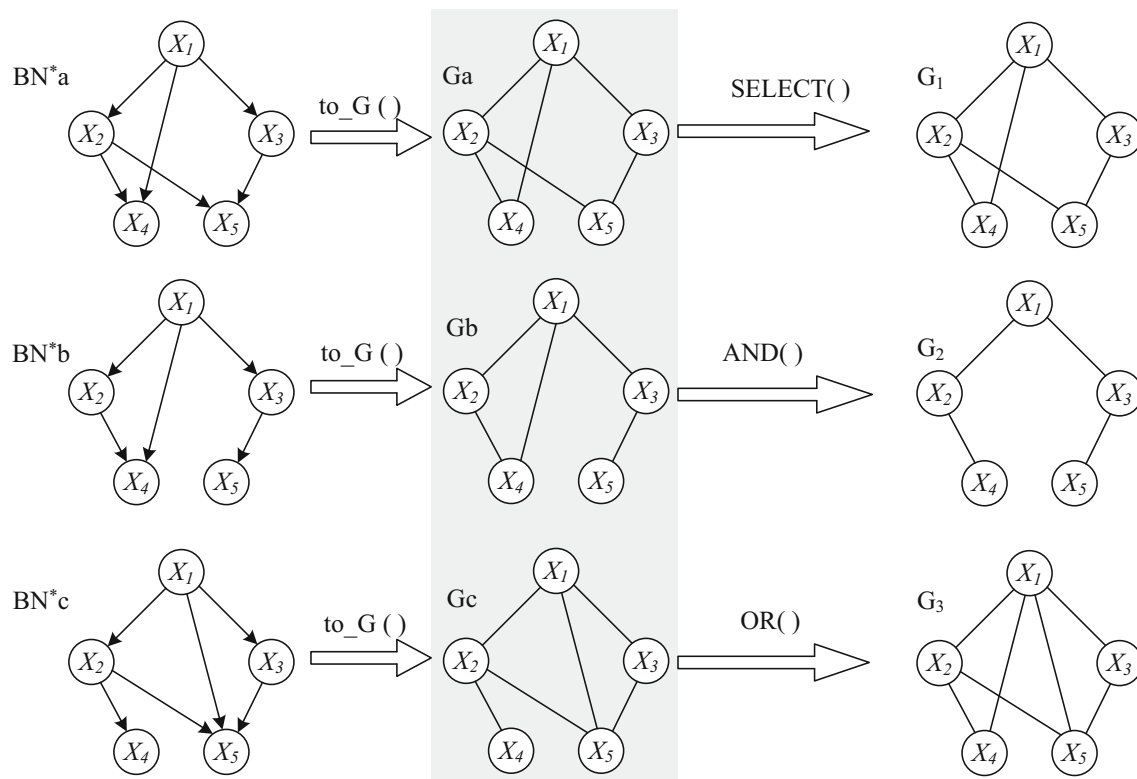
Fig. 3 The framework of structure decomposition learning of complex Bayesian networks based on iterative learning



learning.

Algorithm 2 presents the pseudo-code of the SELECT operator, which aims to extract a high-quality undirected initial graph for the next iterative decomposition learning. Taking the three optimal non-equivalence class Bayesian network structures (BN^*_a, BN^*_b, BN^*_c) preserved from the previous iteration as input, the operator first identifies the

directed acyclic graph DAG with the highest score among the three candidates. Subsequently, the $to_G(DAG)$ function converts this top-scoring DAG into its corresponding undirected skeleton (denoted as G_1) by removing the directionality of all edges, thereby retaining the reliable dependency relationships between nodes encoded in the highest-scoring DAG to facilitate more accurate subsequent decomposition learning.



Note: $\text{Score}(\text{BN}^*a) > \text{Score}(\text{BN}^*b) > \text{Score}(\text{BN}^*c)$

Fig. 4 The process of obtaining undirected graph by three mutation operators

Algorithm 2 SELECT()

Input: $\text{BN}^*a, \text{BN}^*b, \text{BN}^*c$

Output: G_1

//identifying the directed acyclic graph

DAG = BN^*a ,

if $\text{score}(\text{BN}^*b) > \text{score}(\text{DAG})$,

DAG = BN^*b

elseif $\text{score}(\text{BN}^*c) > \text{score}(\text{DAG})$,

DAG = BN^*c

end

//Algorithm 1, converting a DAG into its corresponding undirected skeleton

$G_1 = to_G(\text{DAG})$

(2) AND operator

In the construction of undirected graphs, it is common to obtain a set of pseudo-edges. To refine the graph structure, we employ the AND operator to eliminate these pseudo-edges, ensuring a more accurate representation of the underlying network.

The AND operator is a common way to learn Bayesian network architecture to handle symmetric check problems by treating asymmetric node pairs as false positives and removing the node from each other's PC or MB.

Definition 1 (*AND operator*) For a node X to be adjacent to T in G , the following two statements hold true: (1) X must be in the PC set of T , and (2) T must be in the PC set of X , i.e., $X \in PC_G(T)$ and $T \in PC_G(X)$.

According to the definition1, if there are two nodes in at least one graph without a dependent edge, but in the other graph, it means that the edge is considered as false. Therefore, the edge will be removed from the undirected graph.

As shown in Fig. 4, we extract the skeleton from the three Bayesian network structures of BN^*a , BN^*b , BN^*c , respectively, and get undirected graphs $G(BN^*a)$, $G(BN^*b)$, and $G(BN^*c)$. Then, we could discard some edges that are not common. For example, undirected edges X_1-X_4 , X_2-X_5 , X_1-X_5 , are discarded by AND operator, because they are not common in $G(BN^*a)$, $G(BN^*b)$, and $G(BN^*c)$. A new undirected graph G_2 is obtained by preserving the undirected edges shared by $G(BN^*a)$, $G(BN^*b)$, and $G(BN^*c)$. The shared edges that are preserved are most likely to be true edges, while the ones that are discarded are best likely to be false edges.

(3) OR operator

When constructing an undirected graph, it is common to lose some real edges. In this paper, we could recall them by OR operator.

Like the AND operator, the OR operator is another common way to deal with symmetric check problems by treating an asymmetric node pair as a false negative and adding that node to the other node's PC or MB.

Definition 2 (*OR operator*): For a node X to be not adjacent to T in G , the following two statements hold true: (1) X must not be in the PC of T and (2) T must not be in the PC of X , i.e., $X \notin PC_G(T)$ and $T \notin PC_G(X)$.

According to the Definition 2, if there are two nodes that have a dependent edge in at least one graph and none in the others, it means that the edge is consider as true. Therefore, the edge will be recalled in undirected graph.

As shown in the Fig. 4, we extract the skeleton from the three Bayesian network structures of BN^*a , BN^*b and BN^*c respectively to obtain $G(BN^*a)$, $G(BN^*b)$ and $G(BN^*c)$. Undirected edges X_1-X_4 , X_2-X_5 , X_1-X_5 will be left as true

Algorithm 3 AND()

Input: BN^*a , BN^*b , BN^*c ,
Output: G_2
<pre> n = size(BN^*a) R = zero_matrix(n, n) //extracting consensus edge information for i = 1:n for j = 1:n if $BN^*a[i][j] == 1 \cap BN^*b[i][j] == 1 \cap BN^*c[i][j] == 1$ R[i][j] = 1 else R[i][j] = 0 end end return R //Algorithm 1, converting R into its corresponding undirected skeleton $G_2 = to_G(R)$ </pre>

Algorithm 3 outlines the pseudo-code for the AND operator, designed to generate an undirected initial graph (G_2) for iterative decomposition learning by extracting consensus edge information from three optimal Bayesian network structures (BN^*a , BN^*b , BN^*c). The algorithm stores the consensus edge information in R . And then, $to_G()$ converts R into the undirected skeleton G_2 , preserving only the most reliable edge dependencies shared across all candidate networks for subsequent decomposition learning.

edges in G_3 by OR operator, because they appear in either $G(BN^*a)$, $G(BN^*b)$ and $G(BN^*c)$.

Algorithm 4 presents the pseudo-code for the OR operator, designed to generate an undirected initial graph (G_3) for iterative decomposition learning by aggregating all edge information from three optimal Bayesian network structures (BN^*a , BN^*b , BN^*c). The algorithm stores all edge information present in any of the three candidate networks in R . And then, $to_G()$ converts R into the undirected skeleton G_3 , retaining comprehensive edge dependency information to expand the search space for subsequent decomposition learning.

Algorithm 4 OR()

Input: BN^*a, BN^*b, BN^*c Output: G_3
<pre> n = size(BN^*a) R = zero_matrix(n, n) //aggregating all edge information from BN^*a, BN^*b, BN^*c for i = 1:n for j = 1:n if $BN^*a[i][j] == 1 \cup BN^*b[i][j] == 1 \cup BN^*c[i][j] == 1$ R[i][j] = 1 else R[i][j] = 0 end end end return R //Algorithm 1, converting R into its corresponding undirected skeleton $G_3 = to_G(R)$ </pre>

The SELECT, AND, and OR operators are strategically designed to complement one another in the iterative framework, forming a synergistic mechanism that balances structural reliability, and iterative optimization stability, thereby effectively avoiding local optima in Bayesian network structure learning. Specifically, the AND operator plays a conservative, sparsifying role. By retaining only consensus edges across multiple high-quality BN structures, it filters out noisy or spurious edges and generates a skeleton with highly reliable dependency relationships, but may risk losing potential critical non-consensus edges. In contrast, the OR operator serves as an exploratory, densifying counterpart: it aggregates all edge information from input structures to maximize the coverage of potential dependencies, compensating for the AND operator's conservatism and expanding the iterative search space, though it may introduce redundant edges. The SELECT operator plays a role in memorizing the best structural information: by selecting the top-scoring Bayesian network structure from the previous iteration as the baseline network structure for AND and OR mutation operations with other two suboptimal network structures, it ensures that the iterative update process remains anchored to a high-quality structure and does not lose high-scoring structural information.

Algorithm implementation

Algorithm 5 is the pseudo-code of complex Bayesian network structure decomposition learning based on iterative learning proposed in this paper. It shows that the algorithm has 6 stages, where, the first stage is initialization, and the 2–6 stages are iterative processes.

Firstly, Algorithm 5 randomly generates three Bayesian network structures, and initialize the optimal network. They are with the same nodes. From these structures, we extract the skeletons, resulting in three corresponding undirected initial graphs. The notation to_G(BN) refers to the process of extracting the skeleton from a Bayesian network, in which all the directionality from all edges within the BN removed.

In the iterative process, our algorithm employs three different undirected graph mutation operators to obtain new undirected graphs from three different variation directions. Therefore, three undirected graphs are generated in each iteration. Meanwhile, the three undirected graphs are decomposed and learned in parallel. The second step is decomposing. The three undirected graphs G_1 , G_2 and G_3 are decomposed into three subgraph groups by decompose() algorithm. Where, G_{2m} represents the m th subgraph of undirected graph G_2 . Decompose() is a complex Bayesian network structure decomposition algorithm by a locally-extended overlapping community detection method. The procedure is shown in Algorithm 6. The third step is learning subgraph with subgraph_learn(), which should be accomplished by the global learning algorithm. The global learning algorithm is not the main focus of this study. For convenience, the subgraph_learn() function adopts an advanced traditional global learning algorithm, such as HC, greedy equivalence class search GES [38], K2 algorithm, GA algorithm, BPSO algorithm [39], etc. At this stage, the corresponding small sub-DAGs of all subgraphs are obtained. After obtaining sub-DAGs of three undirected subgraph groups, we combine these sub-DAGs into three complete DAGs, BN_1 , BN_2 and BN_3 in the fourth step. When combining sub-DAGs, there may be some conflicting edges between sub-DAGs. There-

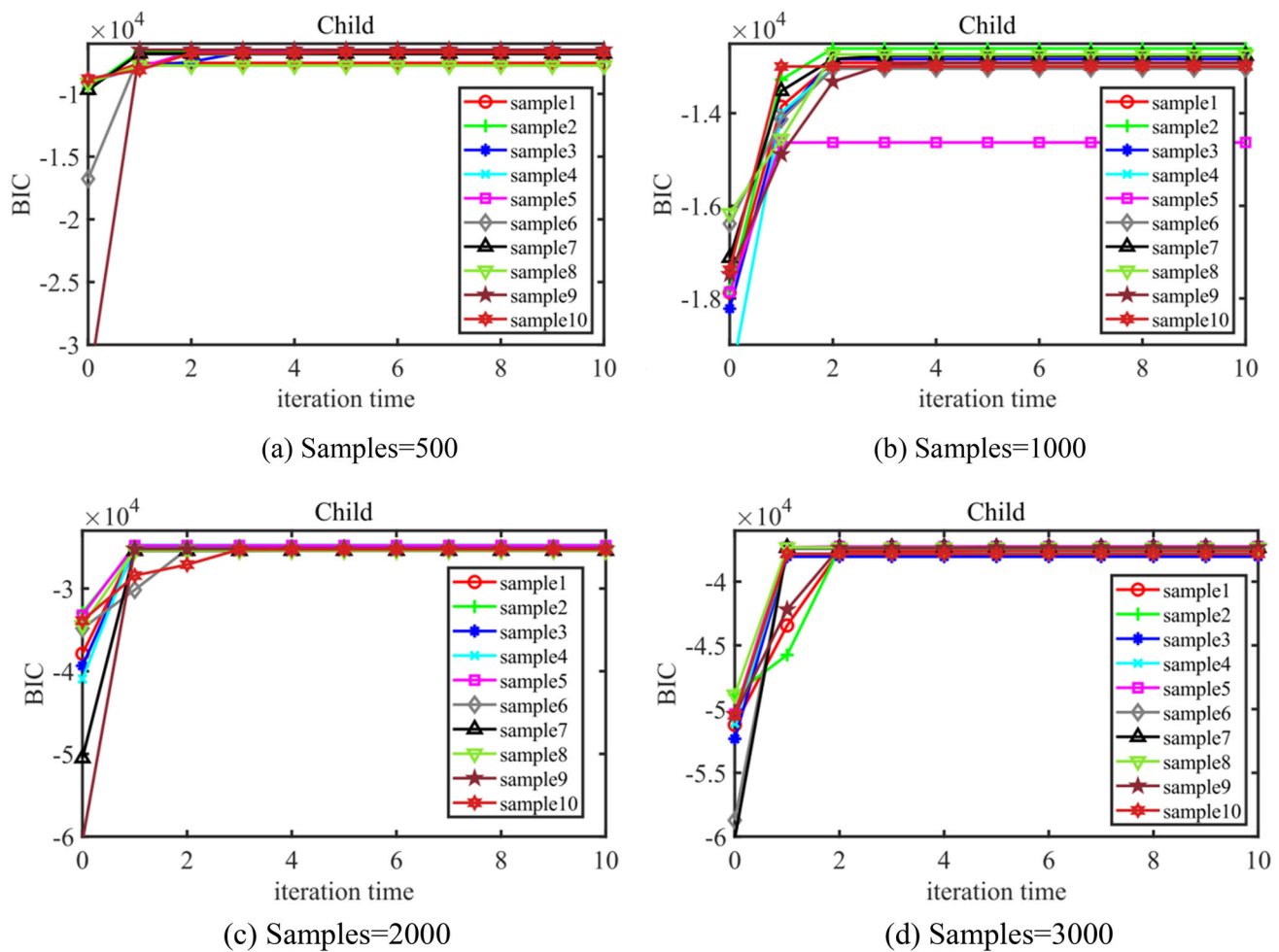


Fig. 5 Iterative learning process on the Child BNs

fore, it needs to determine the structure of these conflicting edges and keep the other structures constant of DAGs. We determine the conflicting structure between sub-DAGs by scoring. By the method, subgraphs are learned and combined into a whole Bayesian network structure. The fifth step is to update the optimal Bayesian network structure obtained in the iteration history. BN_1 , BN_2 , BN_3 are the three Bayesian networks produced by the current iteration. BN^*a , BN^*b , BN^*c are the three optimal network structures obtained in the last iteration. In this step, BN_1 , BN_2 , BN_3 , BN^*a , BN^*b , BN^*c are sorted in descending order of score, and top 3 non-equivalent networks with highest scores are extracted. BN^*a , BN^*b , BN^*c then are updated.

The sixth step is to determine whether the pre-set number of iterative learning has been reached. If not, SELECT, AND and OR operators are executed on BN^*a , BN^*b , and BN^*c to produce three new undirected graphs and proceed to the next iteration. Otherwise, the algorithm exits the iteration and outputs BN^*a as the optimal network. According to the results of the experiment, our proposed algorithm can generally converge to the optimal result after more than four iterations. Therefore, we predetermine $iter = 10$ during the initialization stage.

Algorithm 5 *iter_DSLA()*

Input: Data D of node set X Output: DAG
<pre> //Step1 Initializing Initializing <i>iter</i>, Randomly generate Bayesian network structure: BN_a, BN_b, BN_c; BN*a=BN_a, BN*b=BN_b, BN*c=BN_c; G₁=to_G(BN_a), G₂=to_G(BN_b), G₃=to_G(BN_c) repeat for <i>i</i>=1: <i>iter</i> //Step2 Decomposing {G₁₁,G₁₂,...,G_{1k}}=decompose(G₁); {G₂₁,G₂₂,...,G_{2m}}=decompose(G₂); {G₃₁,G₃₂,...,G_{3n}}=decompose(G₃); //algorithm 6, k, m and n represent the total number of obtained subgraphs. //Step3 Learning subgraph {BN₁₁, BN₁₂,...,BN_{1k}}=subgraph_learn(G₁₁,G₁₂,...,G_{1k}); {BN₂₁, BN₂₂,...,BN_{2m}}=subgraph_learn(G₂₁,G₂₂,...,G_{2m}); {BN₃₁, BN₃₂,...,BN_{3n}}=subgraph_learn(G₃₁,G₃₂,...,G_{3n}); //Step4 Combining BN₁=subgraph_combine(BN₁₁, BN₁₂,...,BN_{1k}); BN₂=subgraph_combine(BN₂₁, BN₂₂,...,BN_{2m}); BN₃=subgraph_combine(BN₃₁, BN₃₂,...,BN_{3n}); //Step5 Updating the optimal network and sorting // Constructing list of networks to be sorted network_list = [BN*a, BN*b, BN*c, BN₁, BN₂, BN₃] // Initializing list to store (score, network) pairs scored_list = empty_list() // Calculating score for each network for network in network_list scored_list.append((score(network), network)) // Sorting networks in descending order of score sorted_list = sort(scored_list) // Extracting top 3 non-equivalent networks with highest scores BN*a = sorted_list[0].network, BN*b = sorted_list[1].network, BN*c = sorted_list[2].network //Step6 Exiting or producing new undirected graphs G₁=SELECT(G(BN*a),G(BN*b), G(BN*c)); G₂=AND(G(BN*a),G(BN*b), G(BN*c)); G₃=OR(G(BN*a),G(BN*b), G(BN*c)); end for BN*=BN*a </pre>

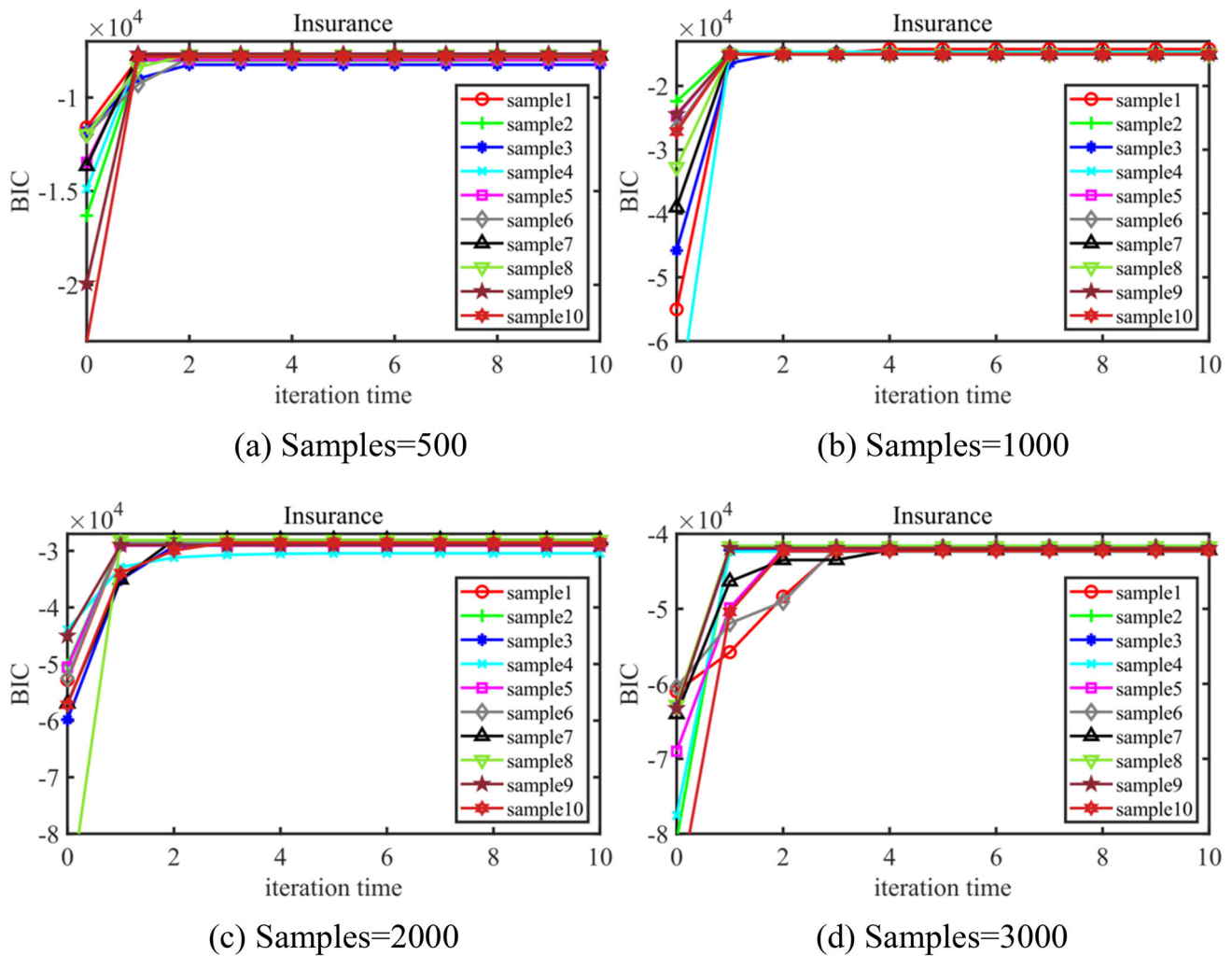


Fig. 6 Iterative learning process on the Insurance BNs

Next, we will focus on introducing Algorithm 6, which is the subroutine in the `iter_DSLA()`. Algorithm 6 is a complex Bayesian network structure decomposition algorithm by locally-extended overlapping community detection [40]. The algorithm has two steps, expansion and merging. The expansion is to find the subgraphs for each node. The merging is combining two subgraphs that overlap too much.

For clarity, the node set V is bifurcated into internal nodes X_{inter} and external nodes X_{exter} in the decomposition process. Where, internal nodes X_{inter} encompass node X_i and its immediate connections, which are grouped into communities. External nodes X_{exter} include node X_i and its neighbors, where a minimum of one node among them does not belong to any community.

In the expansion phase of Algorithm 6, the subgraphs set **groups** starts off as empty and the external nodes set X_{exter} is filled with all nodes in V . When a new subgraph is spotted, the expansion process begins with a node from the external

nodes set X_{exter} serving as the initial seed node **seed**. Starting with the seed node as the foundation of the subgraph, the algorithm looks for the neighboring node that brings the biggest boost to the fitness function and incorporates it into the subgraph. This continues until no further improvements to the fitness value can be made, at which point a new subgraph has been fully expanded from the external nodes set X_{exter} . This new subgraph is then added to the groups and the external nodes set X_{exter} is updated accordingly. Algorithm 6 repeatedly scans for new subgraphs within the external nodes set X_{exter} and transfers new internal nodes from the external node pool to refresh the external nodes set X_{exter} . When the external nodes set X_{exter} become empty, it signifies that all possible subgraphs have been identified. Since nodes connecting to the outside of a subgraph are targeted by other subgraphs as well, there is a natural tendency for subgraphs to overlap with one another.

Algorithm 6 Decompose ()

```

Input:  $G, V, \beta$ 
Output: groups

Step1: expansion

Initialize:  $\mathbf{groups} = \emptyset, X_{\text{exter}} = V$ ;

while  $X_{\text{exter}} \neq \emptyset$  do

     $\mathbf{seed} \leftarrow \arg \max_{X_i \in X_{\text{exter}}} (X_i.\text{degree}), \mathbf{C} \leftarrow \mathbf{seed}, \mathbf{flag1} = \text{ture};$ 

    while  $\mathbf{flag1}$  do

         $X_i, f(X_i) \leftarrow \arg \max_{X_i, f(X_i)} (f(X_i)), X_i \in \text{nb}(\mathbf{C})$  // Formula (3)

        if  $f(X_i) > 0$  then

             $\mathbf{C} \leftarrow \mathbf{C} \cup \{X_i\},$ 

            while  $\exists Y_i \in \mathbf{C}$  such that  $f(Y_i) < 0$  do

                 $\mathbf{C} \leftarrow \mathbf{C} / \{Y_i\}$ 

            end while

        else

             $\mathbf{flag1} = \text{false};$ 

        end if

    end while

     $\mathbf{groups} \leftarrow \mathbf{groups} \cup \mathbf{C}; X_{\text{exter}} = \text{nb}(X_{\text{exter}} / \mathbf{C});$ 

end while

Step2: merging

 $\mathbf{flag} = \text{true}$ 

while  $\mathbf{flag}$  do

    for  $\mathbf{C}_1, \mathbf{C}_2 \in \mathbf{groups}$  do

         $\mathbf{C}_{ov} = \mathbf{C}_1 \cap \mathbf{C}_2; \mathbf{C} = \mathbf{C}_1 \cup \mathbf{C}_2;$ 

        if  $(|\mathbf{C}_{ov}|/|\mathbf{C}_1| > \beta \text{ or } |\mathbf{C}_{ov}|/|\mathbf{C}_2| > \beta)$  then

             $\mathbf{groups} \leftarrow \mathbf{groups} / \mathbf{C}_1; \mathbf{groups} \leftarrow \mathbf{groups} / \mathbf{C}_2; \mathbf{groups} \leftarrow \mathbf{groups} \cup \mathbf{C};$ 

        else

             $\mathbf{flag} = \text{false};$ 

        end if

    end for

end while

Return groups

```

The fitness function [29] is

$$f(\mathbf{C}) = \frac{k_{in}^{\mathbf{C}}}{(k_{in}^{\mathbf{C}} + k_{out}^{\mathbf{C}})^{\alpha}} \quad (3)$$

where, $k_{in}^{\mathbf{C}}$ and $k_{out}^{\mathbf{C}}$ refer to the total internal and external degrees of the nodes within a module $\mathbf{C}$, respectively. The internal degree of a node in a module is the number of edges connecting the node to other nodes within the same module, while the external degree is the number of edges connecting the node to nodes in different modules. α is a positive

real-valued parameter, controlling the size of the communities. A larger value of α would result in larger communities, while a smaller value would lead to smaller, more tightly-knit communities.

In the merging phase of Algorithm 6, it addresses the issue of excessive overlap between certain subgraphs identified in the preceding expansion phase. This redundant overlap can lead to an unnecessary amount of computational repetition during subgraph learning and escalate the effort required for their integration. To mitigate this, the algorithm includes a merging process for these overlapping subgraphs [41]. If the proportion of the shared nodes between two subgraphs exceeds a threshold β in relation to the number of nodes within a single subgraph, it is deemed as excessive overlap. Algorithm 6 amalgamates such overly overlapping subgraphs into a more substantial composite subgraph. β is a hyperparameter that governs the size of the subgraphs and is commonly set to 0.5, indicating that subgraphs are considered to have excessive overlap when the overlapping nodes represent half the number of nodes within a single subgraph.

Theoretical properties of the *iter*-DSL method

For convergence, *iter*-DSL formally terminates at the preset maximum iteration count ($iter = 10$). Its iterative update mechanism (driven by SELECT, AND, OR mutation operators) ensures a monotonic non-decreasing score across iterations. Our method could preclude solution quality deterioration, because the operators retain or aggregate high-quality edge information from top-performing non-equivalent Bayesian network structures of the previous round. Theoretically, *iter*-DSL performance is constrained within a defined range. The lower bound is the score of the initial Bayesian network structure. While the upper bound is governed by subgraph decomposition granularity (regulated by α and β), which balances computational feasibility and structural accuracy. Although structural stability and score convergence are not formal termination criteria, empirical results confirm these stability traits are achieved within 4–5 iterations, validating that $iter = 10$ suffices to converge to a near-optimal solution.

The computational complexity of *iter*-DSL is analyzed based on key parameters: n (network nodes), s (average subgraph size, $s \ll n$), m (sample size), $iter = 10$ (convergent iterations), and $c = 3$ (parallel branches). The total complexity is dominated by decomposition, subgraph learning, and iterative update, formulated as: $O(iter \cdot c \cdot (n \cdot s + \frac{n^2}{s} + n \cdot s^2 \cdot m + n \cdot m))$. The dominant term $O(n \cdot s^2 \cdot m)$ (subgraph learning) is far lower than the $O(n^3 \cdot m)$ of non-decomposition methods (e.g., MMHC) due to $s \ll n$. Compared to non-iterative decomposition methods (SL-MGI, BLMKM), *iter*-DSL

only adds minor iterative overhead while avoiding the computational cost caused by the conditional independence test in the initial graph construction process.

Experiment

Experimental setup

To evaluate the performance of *iter*-DSL method we proposed, we conduct some experiments in this section. We simulated the iterative process and the learning results on three real Bayesian network structures. Then, to test the performance of our algorithm on complex Bayesian network structures, we generated several networks with hundreds or thousands of nodes and conducted experiments on these complex Bayesian network structures. The programs for the proposed algorithm and the comparison algorithms were all compiled using MATLAB software release R2018b, which includes the Bayes Net Toolbox: BNT,¹ Causal Learner.²

In the algorithm, there are three key hyperparameters that need to be explained. α is a community size control parameter in the fitness function that balances community size and structural complexity, and per literature [29], it is set to 1 to avoid overly fragmented or over-expanded subgraphs while retaining local dependency information; β is the overlap threshold for subgraph merging that regulates merging intensity, and supported by literature [40], it is set to 0.5 to prevent under-merging ($\beta > 0.5$) causing redundant computation or over-merging ($\beta < 0.5$) negating decomposition benefits; $iter$ is the maximum number of iterations that defines the upper limit of iterative optimization, and based on experimental observations where convergence is achieved within 4–5 iterations, it is set to 10 to ensure complete updates ($iter < 4$ risks suboptimal results) without unnecessary overhead ($iter > 5$ yields stable performance).

In the experiments, we adopt the HC algorithm to implement both subgraph learning and merging processes. We use F1 [42] and BIC [43] to measure the accuracy. F1 is one of the graphical evaluations criteria corresponding to scoring metrics. A higher F1 score indicates a closer alignment between the learned graph and the true graph, suggesting that the algorithm has effectively captured the underlying structure of the data. Conversely, a lower F1 score suggests that there is a significant discrepancy between the learned and true graph structures, indicating potential issues with the learning algorithm or the data itself. The F1 score is defined as:

$$F1 = \frac{2Pr * Re}{Pr + Re} \quad (4)$$

¹ <https://github.com/bayesnet/bnt>.

² <http://bigdata.ahu.edu.cn/causal-learner>.

Precision (Pr) represents the proportion of correctly identified edges among all the edges that the algorithm has discovered, defined as $Pr = \frac{TP}{TP+FP}$. Recall (Re) represents the proportion of actual edges that are correctly identified by the algorithm among all the true edges that could have been discovered, defined as $Re = \frac{TP}{TP+FN}$. BIC [43] is one of inference-based evaluations criteria generally focusing on how well the learnt Bayesian network fits the observed data. The BIC score is defined as:

$$\text{BIC} = \sum_{i=1}^n \sum_{j=1}^{q_i} \sum_{k=i}^{r_i} m_{ijk} \log \frac{m_{ijk}}{m_{ij*}} - \sum_{i=1}^n \frac{q_i(r_i - 1)}{2} \log m \quad (5)$$

In the experiment, the comparison algorithms we used were SL-MGI [1], BLMKM [16], MMHC [44], PC stable [45] and F2SL [46]. Among them, SL-MGI and BLMKM are two advanced non-iterative structural decomposition learning methods, and MMHC, PC stable and F2SL are non-structural decomposition learning methods. SL-MGI takes advantage of multi-granularity information. In SL-MGI, an undirected independent graph is constructed by CI tests with constrained the number of nodes in condition set. Then, the method decomposed an undirected independence graph by the average-link hierarchical clustering method to realize granulation. Modularity was used to measure the quality of community detection, which naturally transformed the community detection problem into an optimization problem. Finally, these subgraphs correspond to the learned internal structures of the sub-granules. The final network structure is then formulated by utilizing the proposed Construct Link Graph algorithm. BLMKM is a block learning algorithm based on k -means algorithm. The method utilizes MMPC to obtain the undirected independent graph of Bayesian network structures. Then, an improved k -means algorithm is used to block the nodes in BN and possible graph structures based on separated blocks is found by a pruned dynamic programming algorithm. This process aims to obtain potential Bayesian Networks, and the best BN is determined using a scoring function. MMHC is a non-structural decomposition learning method. It begins by establishing the skeleton of the Bayesian network by MMPC, which involves identifying the conditional independence relations among variables. Then, MMHC proceeds with a greedy hill climbing search algorithm that iteratively moves to a neighboring configuration with a higher Bayesian score by adding, removing, and reversing edges. PC stable is a constraint-based Bayesian network structure learning method. It starts by constructing an undirected skeleton through iterative conditional independence tests. Then, PC stable refines the skeleton by orienting edges based on CI test results and causal reasoning rules, gradually transforming the undirected graph into a directed acyclic graph while preserving conditional independence

relationships. F2SL is a two-stage Bayesian network structure learning method that integrates constraint-based and score-based strategies. It first infers the parent-child sets of each variable using constraint-based tests to reduce the search space of potential parent nodes. Then, F2SL employs a score-based greedy search within the restricted PC sets, selecting parent configurations that maximize a Bayesian scoring criterion to construct the final directed acyclic graph.

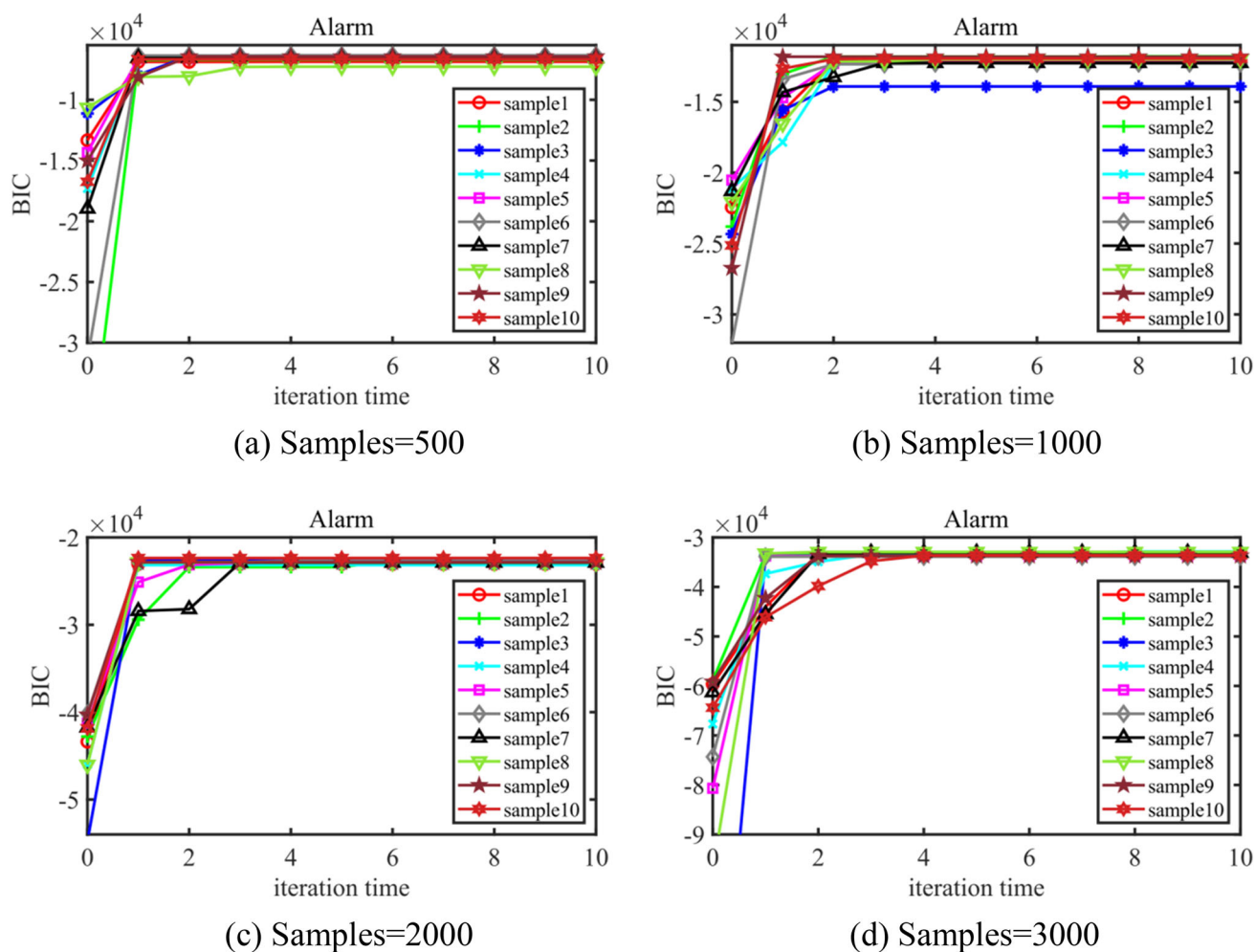
Experiments on the real BNs

To comprehensively characterize the structural complexity of Bayesian Network structures and ensure the general applicability of the proposed *iter*-DSL algorithm, we adopt five widely recognized metrics for Bayesian Network structures description: Number of nodes (network scale), Number of arcs (overall connectivity), Average degree (node connectivity density), Maximum in-degree (parent-child dependency complexity), Average Markov blanket size (local dependency complexity, defined as the average number of parents, children, and children's parents per node). In the experiment, we select three real benchmark Bayesian networks, include Child [47], Insurance [48] and Alarm [49], covering application domains such as pediatric healthcare, car insurance risk evaluation, and patient monitoring medical diagnostic system. The information of three real Bayesian network structures as shown in Table 1. Beyond their varying scales, the three real-world benchmark BNs could effectively represent the structural characteristics of all sparse Bayesian networks, which are prevalent in real-world applications.

As depicted in Figs. 5, 6, and 7, these figures illustrate the iterative learning process of the *iter*-DSL algorithm applied to three different networks: Child, Insurance, and Alarm, using sample sizes of 500, 1000, 2000, and 3000, respectively. Our algorithm commences by learning from the decomposition of random initial graphs, which initially yields relatively low scores. The BIC score rises sharply within the first 2 iterations. The *iter*-DSL rapidly correct suboptimal initial random graph decompositions, and then stabilizes to the optimal value approximately before the fifth iteration, which proved the effective convergence. In most cases, the optimal network can already be obtained for networks with relatively fewer nodes, while for Alarm networks, the optimal network can be obtained in no more than 5 iterations of the learning process. It is evident that as the number of network nodes increases, the number of subgraphs for partitioning also increases, requiring multiple iterations for nodes to find their respective subgraphs before the learning results converge. Meanwhile, the 10 independent run curves all converge to the same optimal BIC score, demonstrating the algorithm's insensitivity to initial random graphs and performance stability. Only in very few cases will *iter*-DSL

Table 1 The information of Child, Insurance, and Alarm BNs

Network	Number of nodes	Number of arcs	Average degree	Maximum in-degree	Average Markov blanket size
Child	20	25	2.5	2	3
Insurance	27	52	3.85	3	5.19
Alarm	37	46	2.49	4	3.51

**Fig. 7** Iterative learning process on the Alarm BNs

reach a local optimum and fail to escape from it, such as in Figs. 5b and 7b.

Tables 2 and 3 present the average learning results from 10 independent runs of *iter*-DSLAs alongside SL-MGI, BLKMM, MMHC, PC stable, and F2SL across the Child, Insurance, and Alarm networks. The optimal results in the tables are highlighted in bold, and the secondary optimal values are highlighted with an underline. For smaller networks with limited data, global methods show stronger data fitting performance. In Table 2, PC stable achieves a notably higher BIC score, especially for Alarm, reflecting its

advantage in fitting global data distributions for small-scale structures. It is worth noting that in these small networks, although *iter*-DSLAs is not the best algorithm for fitting the data, it is still a suboptimal solution. However, according to F1 score in Table 3, *iter*-DSLAs outperforms all competitors from the perspective of structural accuracy. As network complexity and sample size increase, the F1 of structural decomposition learning methods, such as *iter*-DSLAs and BLKMM, surpasses non-structural decomposition learning methods (MMHC, PC stable and F2SL). And on a larger

Table 2 BIC score on Child, Insurance, and Alarm BNs

Network	Samples	<i>iter</i> -DSL	SL-MGI	BLMKM	MMHC	PC stable	F2SL
Child	500	<u>− 6873</u>	− 6911	− 7404	− 6680	− 7297	− 7240
	1000	<u>− 13,045</u>	− 13,352	− 13,583	− 12,944	− 14,753	− 15,524
	2000	− 25,111	− 25,808	<u>− 25,155</u>	− 26,422	− 29,954	− 32,276
	3000	− 37,509	− 38,708	<u>− 37,542</u>	− 38,896	− 46,202	− 46,062
Insurance	500	− 7881	− 8335	<u>− 7888</u>	− 8241	− 8892	− 9159
	1000	<u>− 14,843</u>	− 15,363	− 14,793	− 15,274	− 16,177	− 18,297
	2000	<u>− 28,738</u>	− 29,557	− 28,834	− 29,711	− 27,961	− 34,815
	3000	<u>− 42,003</u>	− 43,120	− 42,035	− 45,033	− 41,473	− 52,253
Alarm	500	− 6639	− 6733	− 6547	− 7446	− 4352	<u>− 4920</u>
	1000	<u>− 12,070</u>	− 12,805	− 12,253	− 13,131	− 10,010	− 16,973
	2000	<u>− 22,788</u>	− 24,189	− 22,867	− 23,706	− 20,764	− 25,642
	3000	− 33,443	− 35,186	− 33,514	− 35,033	− 28,937	<u>− 33,342</u>

Table 3 F1 on Child, Insurance, and Alarm BNs

Network	Samples	<i>iter</i> -DSL	SL-MGI	BLMKM	MMHC	PC stable	F2SL
Child	500	0.704	0.654	0.607	0.768	<u>0.766</u>	0.595
	1000	0.753	0.661	0.625	<u>0.800</u>	0.830	0.500
	2000	0.828	0.680	<u>0.822</u>	0.625	0.772	0.529
	3000	0.872	0.716	<u>0.857</u>	0.658	0.821	0.571
Insurance	500	<u>0.615</u>	0.532	0.617	0.513	0.514	0.441
	1000	0.667	0.629	<u>0.658</u>	0.556	0.635	0.424
	2000	<u>0.709</u>	0.651	0.728	0.591	0.674	0.424
	3000	0.782	0.661	0.782	0.624	0.667	0.400
Alarm	500	<u>0.740</u>	0.674	0.766	0.574	0.659	0.406
	1000	0.798	0.736	<u>0.787</u>	0.650	0.750	0.486
	2000	0.821	0.804	<u>0.806</u>	0.758	0.742	0.515
	3000	0.830	0.811	<u>0.817</u>	0.767	0.748	0.469

Alarm network, the iterative algorithm is consistently superior to BLMKM. While PC stable excels in BIC for small networks, its structural accuracy F1 is consistently outperformed by *iter*-DSL. This contrast highlights that *iter*-DSL balances data fitting and structural accuracy, outperforming global methods in structural recovery even when global methods show stronger short-term data fitting for small networks.

Experiments on more complex Bayesian networks

To test the structural learning performance of our algorithm on more complex Bayesian networks, we conducted experiments on networks with hundreds to thousands of nodes. The experimental setup included a total of six networks, ranging from 100 to 1000 nodes, generated using a random method to create Bayesian network structures. Bayesian networks are typically sparse. The generated network and the real network have similar degree characteristics (average degree: 2–4, maximum in-degree: 7–11). However, the

average Markov blanket range for the generated network is 3.38–8.86. It means that the networks generated are more complex than the three real networks. The specific parameters of the generated networks are detailed in the Table 4.

For the experiments on complex Bayesian networks with hundreds to thousands of nodes, we adopted sample sizes of 1000, 2000, 3000, 4000, and 5000 to ensure reliable estimation of node dependencies in high-dimensional spaces. The experimental results are summarized in Tables 5 and 6, where the “–” symbol indicates that the corresponding algorithm failed to complete the task within 2 h.

In terms of data fitting (Table 5), *iter*-DSL demonstrates competitive performance. It achieves optimal or suboptimal BIC scores in most scenarios, consistently outperforming decomposition-based peers (SL-MGI, BLMKM) and the global method MMHC across all networks and sample sizes. While F2SL leads in BIC scores for some small-scale synthetic networks (e.g., MyBNs1 and MyBNs2

Table 4 The generated network information used in the experiment

Network	Number of nodes	Number of arcs	Average degree	Maximum in-degree	Average Markov blanket size
MyBNs1	100	191	3.82	7	7.98
MyBNs2	200	382	3.82	7	8.59
MyBNs3	400	726	3.63	10	8.59
MyBNs4	600	610	2.03	7	3.38
MyBNs5	800	1525	3.81	11	8.86
MyBNs6	1000	1618	3.24	10	6.75

Table 5 BIC score on the generated network

Network	Samples	<i>iter</i> -DSLA	SL-MGI	BLMKM	MMHC	PC stable	F2SL
MyBNs1	1000	<u>− 67,523</u>	− 69,623	− 69,917	− 70,714	− 68,698	− 62,759
	2000	− 133,941	<u>− 133,592</u>	− 133,848	− 134,989	− 131,038	− 137,774
	3000	− 196,691	<u>− 200,878</u>	− 200,978	− 202,386	− 217,148	− 208,899
	4000	− 267,168	− 264,677	<u>− 264,633</u>	− 266,367	− 270,757	− 252,705
	5000	− 322,064	− 332,147	− 331,883	− 332,817	<u>− 326,663</u>	− 341,372
MyBNs2	1000	− 133,276	<u>− 137,317</u>	− 137,650	− 139,168	− 144,709	− 201,019
	2000	<u>− 269,090</u>	− 270,751	− 271,465	− 273,977	− 281,661	− 241,502
	3000	<u>− 394,500</u>	− 396,961	− 397,844	− 400,363	− 404,696	− 393,725
	4000	− 651,628	− 534,043	− 533,974	− 535,747	<u>− 426,256</u>	− 401,667
	5000	− 657,716	− 660,492	<u>− 659,254</u>	− 664,028	− 686,687	− 699,486
MyBNs3	1000	<u>− 274,988</u>	− 275,029	− 276,100	− 280,355	− 290,668	− 269,582
	2000	− 535,933	<u>− 541,046</u>	− 542,067	− 549,044	− 564,999	− 577,683
	3000	− 788,946	<u>− 795,585</u>	− 795,604	− 800,218	− 815,331	− 896,269
	4000	− 1,059,125	− 1,061,917	<u>− 1,061,518</u>	− 1,067,309	− 1,096,569	− 1,225,999
	5000	− 1,313,785	− 1,326,302	<u>− 1,326,282</u>	− 1,332,635	− 1,443,533	− 1,463,502
MyBNs4	1000	<u>− 395,303</u>	− 395,375	− 398,324	− 405,918	− 388,952	− 400,067
	2000	− 778,576	<u>− 778,728</u>	− 782,217	− 790,416	− 803,666	− 800,642
	3000	<u>− 1,171,737</u>	− 1,170,499	− 1,171,932	− 1,180,727	− 1,191,906	− 1,231,141
	4000	− 1,538,766	− 1,546,175	<u>− 1,546,143</u>	− 1,556,011	− 1,608,904	− 1,618,773
	5000	− 1,916,508	− 1,921,835	<u>− 1,920,753</u>	− 1,930,542	− 1,966,351	− 2,018,933
MyBNs5	1000	− 549,020	<u>− 549,042</u>	− 551,405	− 564,525	− 569,270	− 569,848
	2000	− 1,045,019	<u>− 1,081,980</u>	− 1,087,626	− 1,099,719	− 1,144,226	− 1,141,709
	3000	− 1,552,382	<u>− 1,600,775</u>	− 1,607,132	− 1,612,955	− 1,628,736	− 1,727,441
	4000	− 2,087,029	<u>− 2,127,064</u>	− 2,132,480	− 2,136,822	− 2,207,250	− 2,288,897
	5000	− 2,602,161	− 2,669,295	<u>− 2,669,011</u>	− 2,682,288	− 2,861,051	− 2,933,020
MyBNs6	1000	− 681,161	<u>− 680,394</u>	− 684,884	− 703,951	− 671,473	− 705,827
	2000	− 1,240,595	− 1,332,529	− 1,335,218	− 1,352,642	<u>− 1,304,825</u>	− 1,404,841
	3000	− 1,891,662	<u>− 1,998,210</u>	− 2,000,825	− 2,014,453	−	− 2,133,053
	4000	− 2,603,572	<u>− 2,638,150</u>	− 2,639,501	− 2,651,448	−	− 2,808,420
	5000	− 3,159,845	<u>− 3,279,814</u>	− 3,280,268	− 3,306,039	−	− 3,525,967

Table 6 F1 on the generated network

Network	Samples	<i>iter</i> -DSL _A	SL-MGI	BLMKM	MMHC	PC stable	F2SL
MyBNs1	1000	0.699	0.473	0.487	0.490	0.582	<u>0.642</u>
	2000	0.768	0.570	0.621	0.574	<u>0.752</u>	0.682
	3000	0.784	0.597	0.696	0.600	<u>0.773</u>	0.670
	4000	0.800	0.602	<u>0.718</u>	0.647	0.713	0.679
	5000	0.813	0.626	0.719	0.660	<u>0.776</u>	0.725
MyBNs2	1000	0.583	0.472	0.435	0.442	<u>0.572</u>	0.505
	2000	<u>0.682</u>	0.539	0.564	0.524	0.710	0.592
	3000	0.705	0.567	0.606	0.566	<u>0.684</u>	0.636
	4000	0.737	0.610	<u>0.686</u>	0.650	0.566	0.614
	5000	0.777	0.604	0.695	0.622	<u>0.730</u>	0.604
MyBNs3	1000	0.437	0.425	0.368	0.374	<u>0.519</u>	0.569
	2000	<u>0.577</u>	0.512	0.493	0.462	0.629	0.534
	3000	<u>0.589</u>	0.558	0.570	0.523	0.644	0.438
	4000	<u>0.639</u>	0.582	0.616	0.567	0.658	0.431
	5000	0.693	0.594	0.604	0.594	<u>0.647</u>	0.436
MyBNs4	1000	0.496	<u>0.481</u>	0.359	0.346	0.473	0.345
	2000	0.563	<u>0.547</u>	0.446	0.413	0.515	0.374
	3000	0.608	0.545	0.492	0.451	<u>0.549</u>	0.386
	4000	0.624	<u>0.585</u>	0.531	0.460	0.541	0.394
	5000	0.633	<u>0.588</u>	0.546	0.480	0.567	0.396
MyBNs5	1000	<u>0.425</u>	0.416	0.324	0.331	0.455	0.200
	2000	0.561	0.509	0.486	0.443	<u>0.526</u>	0.246
	3000	0.595	0.552	<u>0.553</u>	0.519	0.552	0.263
	4000	0.646	0.564	<u>0.588</u>	0.554	0.571	0.260
	5000	0.681	0.583	<u>0.633</u>	0.584	0.572	0.274
MyBNs6	1000	0.448	<u>0.432</u>	0.326	0.321	0.349	0.191
	2000	0.571	<u>0.519</u>	0.458	0.427	0.398	0.203
	3000	0.607	<u>0.563</u>	0.544	0.507	–	0.224
	4000	0.701	<u>0.591</u>	0.583	0.534	–	0.209
	5000	0.728	<u>0.614</u>	0.613	0.548	–	0.237

with 100 and 200 nodes), its advantage diminishes as network scale expands (such as MyBNs3 with 300 nodes). On more complex networks with over 300 nodes, our algorithm demonstrated strong competitiveness in data fitting. When the sample size was larger, its robustness was even stronger.

Regarding structural accuracy, *iter*-DSL_A exhibits superior and stable performance across all large-scale networks. As network complexity and sample size increase, F1 score of *iter*-DSL_A consistently leads the highest values among competitors. For MyBNs6, F1 scores of *iter*-DSL_A outperform SL-MGI, BLMKM, MMHC, F2SL and PC stable by a significant margin. At the same time, F1 score of *iter*-DSL_A increases steadily with sample size, reflecting its ability to effectively leverage more data to refine subgraph partitioning

and dependency estimation. Collectively, these findings confirm that *iter*-DSL_A is well-suited for real-world large-scale sparse Bayesian network learning tasks, where computational efficiency and structural accuracy are critical.

Conclusion and future work

In the current structure decomposition learning methods, this process usually completes after sequentially performing the steps of constructing the initial graph, decomposition, subgraph learning, and subgraph merging once. In this paper, we firstly introduce iterative learning into complex Bayesian network structural decomposition learning methods to enhance

the effectiveness of structural decomposition learning. In this method, we have designed a framework for iterative structural decomposition learning. Considering that the skeleton of the learned directed graph is closer to the uncorrelated structure of the real network, this is more advantageous for structural decomposition learning. Therefore, the iterative decomposition learning framework we designed primarily uses the skeleton of the current optimal Bayesian networks as the initial graph for the next iteration of decomposition learning. Additionally, in this paper, we propose three operators, SELECT, AND, OR, to generate new initial graphs in iterative processes to prevent learning from getting stuck in a local optimum. The mutation operators facilitate the generation of new initial graphs by strategically utilizing existing structural information in various manners, thereby enhancing the efficiency of the iterative learning process. Experimental results on real and constructed networks with hundreds or thousands of nodes shows that multiple iterations of learning can effectively improve the quality of the learning network. Our parallel iterative strategy and three special mutation operators enable iterative learning to converge and obtain high-quality Bayesian networks in less than 5 iterations.

While *iter*-DSLA demonstrates superior performance in most scenarios, it may exhibit a key limitation closely related to the core iterative mechanism: the method is highly dependent on the quality and diversity of initial undirected skeletons generated by the three mutation operators. The AND operator may retain only consensus edges, the SELECT operator may select a top-scoring DAG that is similar to the inputs, and the OR operator may aggregate redundant edge information. These three operators may store partially consistent information. Specifically, if the randomly generated initial Bayesian networks have severe structural similarity, or the structural information output by the SELECT, AND, and OR operators is overly consistent. If such consistent information is excessive, the iterative process is prone to stagnation. In such cases, the undirected skeletons learned in consecutive iterations may lack effective updates, failing to introduce new and reliable edge dependency information. This may make the subsequent subgraph decomposition based on these skeletons unreliable (e.g., possibly missing critical edges or retaining false edges in subgraphs), ultimately possibly reducing the accuracy of the learned BN structure. To address this, future work will focus on optimizing the mutation operators with a diversity-aware mechanism, and designing an adaptive monitoring strategy to detect and break iterative stagnation.

Author contributions All authors contributed to the study conception and design. Material preparation, data collection and analysis were performed by Xiaolong Jia, and Hongru Li. The first draft of the manuscript was written by Xiaolong Jia and all authors commented on previous

versions of the manuscript. All authors read and approved the final manuscript.

Funding This work was jointly supported in part by the Natural Science Foundation of China (No. 62073062), and the Natural Science Foundation of Ningxia (2024AAC03334).

Data availability Enquiries about data availability should be directed to the authors.

Declarations

Conflict of interest All the authors of this research paper declare that there is no conflict of interest.

Ethical approval This article does not contain any studies with human participants or animals performed by any of the authors.

Informed consent Informed consent was obtained from all individual participants included in the study.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

1. He CF et al (2021) A multi-granularity information-based method for learning high-dimensional Bayesian network structures. *Cogn Comput*. <https://doi.org/10.1007/s12559-021-09891-0>
2. Scanagatta M, Salmerón A, Stella F (2019) A survey on Bayesian network structure learning from data. *Prog Artif Intell* 8(4):425–439. <https://doi.org/10.1007/s13748-019-00194-y>
3. Yang G, Zhao YL, Gu XH (2021) A novel Bayesian framework with enhanced Principal Component Analysis for chemical fault diagnosis. *IEEE Trans Instrum Meas*. <https://doi.org/10.1109/TIM.2020.3034975>
4. Ma L, Dong J, Peng KX (2018) A practical propagation path identification scheme for quality-related faults based on nonlinear dynamic latent variable model and partitioned Bayesian network. *J Frankl Inst-Eng Appl Math* 355(15):7570–7594
5. Rusek J et al (2021) Score-based Bayesian belief network structure learning in damage risk modelling of mining areas building development. *J Clean Prod*. <https://doi.org/10.1016/j.jclepro.2021.126528>
6. Chickering DM, Heckerman D, Meek C (2004) Large-sample learning of Bayesian networks is NP-hard. *J Mach Learn Res* 5:1287–1330
7. Liu H et al (2017) A new hybrid method for learning Bayesian networks: separation and reunion. *Knowl Based Syst* 121:185–197. <https://doi.org/10.1016/j.knosys.2017.01.029>

8. Kitson NK et al (2023) A survey of Bayesian network structure learning. *Artif Intell Rev* 56(8):8721–8814. <https://doi.org/10.1007/s10462-022-10351-w>
9. Spirtes P, Glymour C (2016) An algorithm for fast recovery of sparse causal graphs. *Soc Sci Comput Rev* 9(1):62–72. <https://doi.org/10.1177/089443939100900106>
10. Cheng J et al (2002) Learning Bayesian networks from data: an information-theory based approach. *Artif Intell* 137(1–2):43–90
11. Tabar VR et al (2018) Finding a set of candidate parents using dependency criterion for the K2 algorithm. *Pattern Recognit Lett* 111:23–29. <https://doi.org/10.1016/j.patrec.2018.04.019>
12. Zhang X et al (2018) Differential-evolution-based coevolution ant colony optimization algorithm for Bayesian network structure learning. *Algorithms Basel*. <https://doi.org/10.3390/a11110188>
13. Contaldi C, Vafaee F, Nelson PC (2019) Bayesian network hybrid learning using an elite-guided genetic algorithm. *Artif Intell Rev* 52(1):245–272
14. Liu K et al (2021) An improved particle swarm optimization algorithm for Bayesian network structure learning via local information constraint. *IEEE Access* 9:40963–40971
15. Huang J, Zhou Q (2021) Partitioned hybrid learning of Bayesian network structures. *Mach Learn* 111:1695–1738
16. Li X, Gao X, Wang C (2020) A novel BN learning algorithm based on block learning strategy. *Sensors (Basel)*. <https://doi.org/10.3390/s20216357>
17. Dai J, Ren J, Du W (2020) Decomposition-based Bayesian network structure learning algorithm using local topology information. *Knowl-Based Syst*. <https://doi.org/10.1016/j.knosys.2020.105602>
18. Wang Manxi WL (2018) Finding community structure of Bayesian networks by improved K-means algorithm. In: 2018 3rd IEEE international conference on image, vision and computing
19. Xie X, Geng Z, Zhao Q (2006) Decomposition of structural learning about directed acyclic graphs. *Artif Intell* 170(4–5):422–439. <https://doi.org/10.1016/j.artint.2005.12.004>
20. Olesen KG, Madsen AL (2002) Maximal prime subgraph decomposition of Bayesian networks. *IEEE Trans Syst Man Cybern Part B* 32:21–31
21. Geng Z, Wang C, Zhao Q (2005) Decomposition of search for v-structures in DAGs. *J Multivar Anal* 96(2):282–294. <https://doi.org/10.1016/j.jmva.2004.10.012>
22. Xie X, Zhi G (2008) A recursive method for structural learning of directed acyclic graphs. *J Mach Learn Res* 9:459–483
23. Chaudhary M, Ranshous R, Samatov NF (2017) A community-driven graph partitioning method for constraint-based causal discovery. In: The sixth international conference on complex networks and their applications. https://doi.org/10.1007/978-3-319-72150-7_21
24. Jung S, Lee KH, Lee D (2007) Enabling large-scale Bayesian network learning by preserving intercluster directionality. *IEICE Trans Inf Syst* E90d(7):1018–1027
25. Shuohao L et al (2014) A graph partitioning approach for Bayesian network structure learning. In: The 33rd Chinese control conference, Nanjing, China
26. Gu JY, Zhou Q (2020) Learning big Gaussian Bayesian networks: partition, estimation and fusion. *J Mach Learn Res* 21:1–31
27. Gregory S (2007) An algorithm to find overlapping community structure in networks. In: *Knowledge discovery in databases: PKDD 2007*. Springer, Berlin Heidelberg
28. Palla G et al (2005) Uncovering the overlapping community structure of complex networks in nature and society. *Nature* 435(7043):814–818. <https://doi.org/10.1038/nature03607>
29. Lancichinetti A, Fortunato S, Kertész J (2009) Detecting the overlapping and hierarchical community structure of complex networks. *New J Phys*. <https://doi.org/10.1088/1367-2630/11/3/033015>
30. Yikun Zhang YL, Liu J (2018) Learning Bayesian network structure by self-generating prior information—the two-step clustering-based strategy. In: *AAAI workshops*
31. Chen J et al (2019) Causal analysis of learning performance based on Bayesian network and mutual information. *Entropy*. <https://doi.org/10.3390/e21111102>
32. Qi XL et al (2021) Mutual-information-inspired heuristics for constraint-based causal structure learning. *Inf Sci* 560:152–167
33. Morota G et al (2012) An assessment of linkage disequilibrium in Holstein cattle using a Bayesian network. *J Anim Breed Genet* 129(6):474–487
34. Kumari P et al (2020) Root cause analysis of key process variable deviation for rare events in the chemical process industry. *Ind Eng Chem Res* 59(23):10987–10999
35. Zalik KR, Zalik M (2023) Density-based entropy centrality for community detection in complex networks. *Entropy*. <https://doi.org/10.3390/e25081196>
36. Yan C, Zhou S (2020) Effective and scalable causal partitioning based on low-order conditional independent tests. *Neurocomputing* 389:146–154. <https://doi.org/10.1016/j.neucom.2020.01.021>
37. Li Y et al (2015) An algorithm for learning the skeleton of large Bayesian network. *Int J Artif Intell Tools*. <https://doi.org/10.1142/s0218213015500128>
38. Puerta JM et al (2019) Structural fusion/aggregation of Bayesian networks via Greedy equivalence search learning algorithm. In: *Symbolic and quantitative approaches to reasoning with uncertainty*. pp 432–443
39. Guo H, Li H (2020) An efficient Bayesian network structure learning algorithm using the strategy of two-stage searches. *Intell Data Anal* 24(5):1087–1106. <https://doi.org/10.3233/ida-194844>
40. Jia XL, Li HR (2025) Structural decomposition-based learning of large Bayesian networks for detecting conditionally independent overlapping superstructure communities. *Appl Intell*. <https://doi.org/10.1007/s10489-025-06601-3>
41. Jiao P et al (2022) A survey on role-oriented network embedding. *IEEE Trans Big Data* 8(4):933–952. <https://doi.org/10.1109/tbdata.2021.3131610>
42. Kitson NK, Constantinou AC, Guo Z, Liu Y, Chobtham K (2021) A survey of Bayesian network structure learning. <https://doi.org/10.48550/arXiv.2109.11415>
43. Schwarz G (1978) Estimating the dimension of a model. *Ann Stat* 6(2):461–464 (4.)
44. Tsamardinos I, Brown LE, Aliferis CF (2006) The max-min hill-climbing Bayesian network structure learning algorithm. *Mach Learn* 65(1):31–78
45. Colombo D, Maathuis MH (2014) Order-independent constraint-based causal structure learning. *J Mach Learn Res* 15:3741–3782
46. Yu K et al (2024) Feature selection for efficient local-to-global Bayesian network structure learning. *ACM Trans Knowl Discov Data* 18(2):27
47. Spiegelhalter DJ, Cowell RG (1992) Learning in probabilistic expert systems. *Bayesian Stat* 4:447–465
48. Binder J et al (1997) Adaptive probabilistic networks with hidden variables. *Mach Learn* 29(2):213–244. <https://doi.org/10.1023/A:1007421730016>
49. Beinlich IA et al (1989) The ALARM monitoring system: a case study with two probabilistic inference techniques for belief networks. In: *AIME 89*. Springer, Berlin, Heidelberg